

Claude Code Agent Teams

オーケストレーター 使用例ガイド

Orchestrator Usage Examples & Patterns

本ドキュメントは Claude Code の Agent Teams 機能を使って開発チームを自動連携させるオーケストレーターの具体的な使用例を、シーン別・パターン別にまとめたガイドです。日常の開発フローに合わせて適切なパターン（P1～P5）を選択することで、品質を保ちながらトークン消費を最適化できます。

目次

1. チーム構成と役割	p. 2
2. 起動パターン早見表（P1～P5）	p. 3
3. セットアップ方法	p. 4
4. 使用例 - 機能開発フロー	p. 5
5. 使用例 - PRレビュー	p. 6
6. 使用例 - リリースゲート	p. 8
7. 使用例 - 新機能計画	p. 9
8. 使用例 - インシデント対応	p. 10
9. 使用例 - バグ修正	p. 11
10. 使用例 - 追加シーン集	p. 12
11. パターン選択フローチャート	p. 14
12. よくある質問	p. 15

1. チーム構成と役割

dev-team には以下の8役割 + 5つのオーケストレーターが含まれます。

役割エージェント一覧

役割	フォルダ	主な責任	単体コマンド例
セキュリティ責任者	セキュリティ責任者/	脆弱性・認証・暗号化	/security-audit
技術責任者	技術責任者/	設計・コード品質・性能	/code-review-tech
法務責任者	法務責任者/	ライセンス・法規制・個人情報	/license-check
経理責任者	経理責任者/	コスト・予算・ROI	/cost-estimate
プロジェクト責任者	プロジェクト責任者/	進捗・リスク・スケジュール	/risk-assessment
QA責任者	QA責任者/	テスト品質・バグ管理	/qa-review
DevOps責任者	DevOps責任者/	インフラ・CI/CD・監視	/deploy-check
UIデザイナー	UIデザイナー/	UI/UX・アクセシビリティ	/ui-review

オーケストレーター一覧

スキル名	コマンド	起動する役割数	用途
orchestrator	/orchestrator	自動判断（1～8）	汎用・自動振り分け
pr-review	/pr-review	4（P3基準）	PRレビュー自動実行
release-gate	/release-gate	6（P2基準）	リリース承認ゲート
feature-planning	/feature-planning	8（P1基準）	新機能の計画・見積もり
incident-response	/incident-response	4～5（緊急）	本番障害の緊急対応

各役割が使用するスキル

役割	使用スキル
セキュリティ責任者	security-audit / vulnerability-check / code-review-security / security-incident
技術責任者	architecture-review / tech-debt-review / code-review-tech / performance-review
法務責任者	license-check / compliance-review / privacy-review / contract-review
経理責任者	cost-estimate / budget-review / resource-cost / roi-analysis
プロジェクト責任者	project-status / sprint-planning / risk-assessment / milestone-review
QA責任者	test-plan / qa-review / bug-triage
DevOps責任者	deploy-check / infra-review / monitoring-setup
UIデザイナー	ui-review / accessibility-check / design-system-review / prototype-review

2. 起動パターン早見表（P1～P5）

トークン消費を最適化するため、状況に応じて5段階のパターンを使い分けます。

パターン	名称	エージェント数	トークン	コマンド	推奨場面
P1	フルチーム	8	★★★★★	/orchestrator	大型リリース・新規プロダクト
P2	リリース前チェック	6	★★★★☆	/release-gate	通常スプリントリリース
P3	開発レビュー	4	★★★☆☆	/pr-review	機能追加・UI変更のPR
P4	コアレビュー	2	★★☆☆☆	(手動指示)	バグ修正・リファクタリング
P5	クイックチェック	1	★☆☆☆☆	(手動指示)	Hotfix・設定変更・タイポ

パターン別 起動エージェント対応表

役割	P1	P2	P3	P4	P5
セキュリティ責任者	✓	✓	✓	—	—
技術責任者	✓	✓	✓	✓	✓
法務責任者	✓	✓	条件	—	—
経理責任者	✓	—	—	—	—
プロジェクト責任者	✓	✓	—	—	—
QA責任者	✓	✓	✓	✓	—
DevOps責任者	✓	✓	条件	—	—
UIデザイナー	✓	—	✓	—	—

条件 = 新規ライブラリ追加(法務) / インフラ変更(DevOps) がある場合のみ追加

3. セットアップ方法

ステップ1: スキルをプロジェクトに配置

Claude Code がスキルを認識するには、プロジェクトルートの `.claude/skills/` に配置します。

```
bash
```

```
# skills_hinagata リポジトリをクローン済みの場合
cd <あなたのプロジェクトルート>

# dev-team フォルダをコピー
cp -r c:/Users/user/cursor_prg/skills_hinagata/dev-team .claude/skills/dev-team

# または シンボリックリンクで共有（更新が自動反映される）
mkdir -p .claude/skills
ln -s c:/Users/user/cursor_prg/skills_hinagata/dev-team .claude/skills/dev-team
```

ステップ2: Claude Code を再起動

配置後に Claude Code（CLI または VSCode 拡張）を再起動するとスキルが認識されます。

```
bash
```

```
# CLI の場合: セッションを再起動
claude

# スキルが認識されたか確認 - / を入力するとリストが表示される
```

ステップ3: 動作確認

Claude Code への入力

```
# 最小パターン（P5）で動作確認
/orchestrator テスト: このプロジェクトの README を技術責任者としてレビューしてください

# または直接スキルを呼ぶ
/security-audit
```

フォルダ構成（配置後）

ディレクトリ構成

```
your-project/
├── .claude/
│   └── skills/
│       ├── dev-team/ ← ここに配置
│           ├── PATTERN_P1_フルチーム.md
│           ├── PATTERN_P2_リリース前チェック.md
│           ├── ... (P3~P5)
│           ├── オーケストレーター/SKILL.md
│           ├── PRレビュー/SKILL.md
│           └── セキュリティ責任者/
```

| |—— 技術責任者/
| |—— ... (各役割)
|—— src/ ... (プロジェクトのソースコード)

4. 使用例 - 機能開発フロー全体像

スプリントの流れに沿ったオーケストレーターの使いどころを示します。

スプリント全体での使用タイミング

フェーズ	タイミング	使用コマンド	パターン
① 計画	スプリント開始前・新機能検討時	/feature-planning	P1
② 開発中	機能追加PRのマージ前	/pr-review	P3
② 開発中	バグ修正PRのマージ前	P4手動指示	P4
② 開発中	タイポ・設定変更PRのマージ前	P5手動指示	P5
③ リリース	本番デプロイ前	/release-gate	P2
③ リリース	大型リリース（四半期）	/orchestrator	P1
④ 運用	本番障害発生時	/incident-response	緊急

具体的な1スプリントの流れ

1スプリントの使用例

【月曜：スプリント計画】

/feature-planning ユーザーが商品をお気に入り登録できる機能を追加したい

→ 全8役割が並列評価 → 工数・コスト・リスク・法務チェック結果を統合

【火～木：開発中】

/pr-review PR#101 ← お気に入り機能の実装PR（機能追加）

→ 技術・セキュリティ・QA・UIの4役割がレビュー

P4手動指示：PR#102のバグ修正（軽微）

→ 技術責任者・QA責任者の2役割のみ

【金曜：リリース】

/release-gate v2.5.0

→ 6役割がGO/NO-GO判定 → 全員GOでデプロイ承認

5. 使用例 - PRレビュー（パターン別）

例1: 機能追加PR - P3 開発レビュー（4エージェント）

Google ログイン機能を追加したPRのレビューを依頼するケース。

Claude Code への入力

```
/pr-review PR#42 GoogleOAuth認証の追加
```

Claude が自動で実行する処理:

- ・ `git diff main...feature/google-oauth` を取得
- ・ 技術責任者・セキュリティ責任者・QA責任者・UIデザイナー を並列起動
- ・ 全員の結果を収集して統合レポートを生成

```
## PR統合レビューレポート PR#42 GoogleOAuth認証の追加
```

役割	判定	主な指摘
技術責任者	[OK] Approve	-
セキュリティ責任者	[!] Comment	state パラメータに CSRF 対策を追加推奨
QA責任者	[NG] Request Changes	認証失敗時のテストケースが不足
UIデザイナー	[OK] Approve	-

最終判定: マージ不可

ブロッカー: QA責任者の指摘（失敗系テスト追加が必須）

任意改善: セキュリティ責任者のCSRF対策コメント

例2: バグ修正PR - P4 コアレビュー（2エージェント）

カート画面の合計金額計算バグを修正したPRのケース。

Claude Code への入力（P4指示）

以下のPRを P4 コアレビュー（技術責任者とQA責任者のみ）でレビューしてください。

PR: #55 カート合計金額の計算バグ修正

変更内容: 消費税の丸め処理を `Math.floor` から `Math.round` に変更（3行変更）

新規ライブラリなし / 認証ロジック非変更 / UI変更なし

```
## P4 コアレビュー結果 PR#55
```

役割	判定	指摘
技術責任者	[OK] Approve	意図通りの修正。変更箇所は最小限で適切
QA責任者	[OK] Approve	既存テストがカバー済み。追加不要

最終判定: マージ可

例3: タイポ修正 - P5 クイックチェック（1エージェント）

Claude Code への入力（P5指示）

以下の変更を P5 クイックチェック（技術責任者のみ）でレビューしてください。

PR: #58 README のタイプ修正
変更内容: docs/README.md の誤字2箇所を修正 (2行変更)
コード変更なし

## P5 クイックチェック結果 PR#58			
役割	判定	指摘	
技術責任者	[OK] Approve	問題なし	
最終判定: 即時マージ可			

6. 使用例 - リリースゲート

例4: 通常スプリントリリース - P2 (6エージェント)

Claude Code への入力

```
/release-gate v2.5.0
```

起動される役割 (P2: 経理責任者・UIデザイナーを除く):

- ・ セキュリティ責任者 → vulnerability-check (未解決Critical/Highがないか)
- ・ 技術責任者 → code-review-tech (P0バグゼロか確認)
- ・ 法務責任者 → compliance-review (法規制への準拠)
- ・ プロジェクト責任者 → milestone-review (マイルストーン達成確認)
- ・ QA責任者 → test-plan (全テストパス・P0/P1バグゼロ)
- ・ DevOps責任者 → deploy-check (デプロイ手順・切り戻し計画)

```
## リリースゲート判定 v2.5.0
```

役割	結果	備考
セキュリティ責任者	[OK] GO	Critical/High 脆弱性 0件
技術責任者	[OK] GO	P0バグ 0件
法務責任者	[OK] GO	ライセンス問題なし
プロジェクト責任者	[OK] GO	スプリントゴール達成
QA責任者	[OK] GO	全自動テストパス
DevOps責任者	[!!] 条件付きGO	切り戻し手順書の更新が必要

最終判定: [!!] 条件付きGO

条件: DevOps責任者が切り戻し手順書を更新後にリリース実行

例5: 緊急 Hotfix リリース (簡略版)

本番障害の緊急修正時は3役割のみで迅速にリリースし、事後に完全レビューを実施します。

Claude Code への入力 (緊急Hotfix)

緊急Hotfixです。以下を3役割 (セキュリティ・技術・DevOps) のみで即時リリース判定してください。

修正内容: PR#99 決済エラー修正 (nullチェック追加 2行)

影響: 本番で決済が5%失敗している (発生から30分経過)

緊急リリース判定: [OK] GO (3役割承認)

事後対応: 次営業日に P2 リリース前チェックで事後レビューを実施すること

7. 使用例 - 新機能計画

例6: 新機能の計画・見積もり - P1 フルチーム (8エージェント)

月額サブスクリプション課金機能を新規追加するかどうかを検討するケース。

Claude Code への入力

/feature-planning 月額サブスクリプション課金機能を追加したい。
ユーザーがプレミアムプランに登録し、毎月自動で課金される仕組みを実装する。
決済は Stripe を使用予定。

機能開発計画書: サブスクリプション課金機能

観点	評価	主要事項
技術的実現性	[OK] 可	Stripe SDK で実装可能
セキュリティ	[!!!] 要検討	PCI DSS 準拠・Webhook署名検証が必須
品質保証	[OK] 可	Stripe Test Mode でE2E可能
法的コンプライアンス	[!!!] 要確認	特商法の表記・解約手続き要件の確認
コスト妥当性	[OK] 可	Stripe手数料 3.6%込みで採算見込み
スケジュール	[OK] 可	3人月で実装可能
インフラ	[OK] 可	既存インフラで対応可能
UI/UXデザイン	[OK] 可	料金プラン画面の新規作成が必要

推奨判定: 条件付き推奨

条件: 法務責任者が特商法・解約要件を確認後に開発着手

開発工数見積もり: 3人月 / 初期費用: ¥3,600,000

例7: 軽微な機能追加の計画 - P3相当 (手動指定)

既存のプロフィール画面にアバター画像アップロード機能を追加するケース (小規模)。

Claude Code への入力 (役割を手動絞り込み)

以下の機能追加について、技術責任者・セキュリティ責任者・UIデザイナーの
3役割で評価してください (小規模機能のため経理・法務・PJは不要)。

機能: プロフィール画面にアバター画像アップロード機能を追加

仕様: JPEG/PNG、最大2MB、S3に保存、既存ユーザーページに表示

技術責任者: [OK] 実装可。S3プリサインドURLパターン推奨

セキュリティ責任者: [!!!] ファイル種別検証 (MIMEタイプ) をサーバー側で必ず実施

UIデザイナー: [OK] プレビュー表示とトリミングUIを追加推奨

推奨判定: 開発推奨 (セキュリティ指摘を実装方針に含めること)

8. 使用例 - インシデント対応

例8: 本番障害（サービス停止） - 緊急召集

Claude Code への入力

```
/incident-response ログイン画面が500エラーを返しており全ユーザーがログイン不可。  
発生時刻: 14:32。直近デプロイ: 13:45 (v2.4.1)。  
エラーログ: DatabaseConnectionPool exhausted
```

起動される役割（並列）:

- ・ DevOps責任者（最優先） → サービス状態確認・切り戻し判断
- ・ 技術責任者 → v2.4.1 の変更内容とDBプール設定を調査
- ・ セキュリティ責任者 → 攻撃起因の可能性確認・証跡保全
- ・ プロジェクト責任者 → ステークホルダーへの第一報

T+0分: /incident-response 実行 → 4役割が並列起動

T+8分: DevOps責任者「v2.4.1 の DB プール設定が誤って max:5 に縮小されている」

T+10分: 技術責任者「PR#88 の設定変更が原因。v2.4.0 にロールバック推奨」

T+12分: プロジェクト責任者「ステークホルダーへ第一報送信済み」

T+15分: DevOps責任者「v2.4.0 へのロールバック完了。サービス復旧確認」

根本原因: DBプール設定の誤変更

再発防止: 設定値変更時のレビュー必須化 / 設定値の上下限アラート追加

例9: 情報漏洩の疑い - セキュリティインシデント

Claude Code への入力

```
/incident-response 不審なAPIアクセスを検知。特定IPから1時間で10万件の  
ユーザー情報エンドポイントへのアクセスがあった。個人情報漏洩の可能性あり。
```

このケースでは法務責任者も追加で起動（個人情報漏洩の疑いがあるため）:

- ・ DevOps責任者 → 対象IPのアクセス遮断・ログ保全
- ・ セキュリティ責任者 → 攻撃手法・認証バイパスの有無調査
- ・ 技術責任者 → レートリミット・認証の実装確認
- ・ 法務責任者 → 個人情報保護法の72時間以内報告義務の確認
- ・ プロジェクト責任者 → 経営層・広報への報告

9. 使用例 - バグ修正・技術的負債

例10: 優先度の高いバグ - P4 コアレビュー

Claude Code への入力 (P4 + セキュリティ追加)

P4 コアレビューをお願いします。

PR#77: 検索機能でXSSが発生する脆弱性の修正

変更: 検索クエリのエスケープ処理を追加 (5行変更)

※ セキュリティ修正のため、セキュリティ責任者を追加してください

P4 基本 + セキュリティ責任者を条件追加した3役割構成:

技術責任者: [OK] Approve エスケープ処理の実装は適切

QA責任者: [OK] Approve 既存のXSSテストがパス。追加テストも十分

セキュリティ責任者: [OK] Approve DOMPurify の使用は正しい選択

最終判定: マージ可 (緊急マージ推奨)

例11: 技術的負債の解消計画

Claude Code への入力 (単体スキル直接呼び出し)

以下のリファクタリングについて、技術責任者に技術的負債レビューを依頼してください。

対象: src/utils/ 配下の重複コード群 (約300行)

内容: jQuery依存コードをVanilla JSへ移行

スキル直接呼び出し

または直接スキルを呼ぶ

/tech-debt-review src/utils/

技術的負債レポート: src/utils/

Type A (意図的・返済予定): ajax-helper.js → fetch APIへの移行 (工数: 0.5日)

Type C (非意図的): dom-utils.js の jQuery 依存 12箇所 (工数: 1日)

Type D (ビット腐敗): legacy-ie-polyfill.js → IE非サポートのため完全削除可

推奨返済順: Type D削除 → Type C移行 → Type A移行

合計工数見積もり: 1.5日

10. 使用例 – 追加シーン集

例12: 新規ライブラリ導入の審査

Claude Code への入力

以下のライブラリ追加について、法務責任者とセキュリティ責任者に審査を依頼します。

追加予定ライブラリ: `moment.js` → `date-fns` への移行 (MITライセンス)

理由: `moment.js` がメンテナンス終了のため

法務責任者: [OK] 承認 `date-fns` は MIT ライセンス。著作権表示のみ必要

セキュリティ責任者: [OK] 承認 既知の CVE なし。メンテ活発

判定: 導入承認

例13: インフラ構成変更のレビュー

Claude Code への入力 (単体スキル)

`/infra-review ECS Fargate` への移行計画をレビューしてください。

現在EC2 `t3.medium` 3台 → ECS Fargate (最小1タスク、最大10タスク) への変更。

DevOps責任者: [!!!] 条件付き承認

- 可用性: マルチAZ設定を必ず有効化すること
- コスト: 常時稼働コスト試算が必要 (経理責任者と連携推奨)
- セキュリティ: タスクロールの最小権限設定を確認

例14: アクセシビリティ監査

Claude Code への入力 (単体スキル)

`/accessibility-check src/components/CheckoutForm.tsx`

決済フォームのWCAG 2.1 AA準拠を確認してください。

UIデザイナー: 準拠レベル A (AA未達)

Critical (A基準違反):

- エラーメッセージが赤色のみで伝達 (色覚異常対応なし)
- カードナンバー入力にラベルが未関連付け (`aria-label` なし)

High (AA達成に必要):

- 送信ボタンのコントラスト比 3.1:1 (基準: 4.5:1)

例15: コスト最適化の相談

Claude Code への入力 (単体スキル)

`/resource-cost` 現在のAWSリソース構成のコスト最適化を分析してください。

月額コスト: ¥180,000。RDS `t3.large` が常時80%以上稼働中。

経理責任者: コスト最適化提案

即時削減（リスクなし）：

- 開発環境RDS：営業時間外停止 → ¥12,000/月削減

中期削減（検証後）：

- RDS t3.large → t3.medium：¥8,000/月削減（負荷テスト後）

最適化後予測：¥160,000/月（削減率 11%）

例16：プライバシーポリシーの更新審査

Claude Code への入力（単体スキル）

/privacy-review 新しいプライバシーポリシー（draft）をレビューしてください。

変更点：アクセス解析ツールを GA4 に変更。Cookie の収集範囲が拡大。

法務責任者：[!!] 要対応

- Cookie同意バナーの設置が必要（改正電気通信事業法）
- GA4のデータ保持期間をポリシーに明記すること
- オプトアウト手順の追記が必要

例17：スプリントレビュー前の確認

Claude Code への入力（単体スキル）

/project-status スプリント12の完了状況を確認してください。

完了タスク：8/10。残2タスクは進行中。

プロジェクト責任者：ステータスレポート

全体ステータス：[!!] 注意（残2タスクの完了リスク）

今週の完了：8タスク / 予定：10タスク（達成率 80%）

課題：決済API統合テストが予想より時間がかかっている

対応：明日の午前中に技術責任者と設計見直しMTGを設定推奨

11. パターン選択フローチャート

以下のフローに従ってパターンを選択してください。

パターン選択フローチャート

今日のタスクは何ですか？

```
├─ 新機能を作るか検討したい
│   └─ /feature-planning [機能概要] → P1 (8エージェント)
│
├─ PRレビューを依頼したい
│   └─ 機能追加・UI変更あり
│       └─ /pr-review [PR番号] → P3 (4エージェント)
│       └─ バグ修正・リファクタリングのみ
│           └─ セキュリティ関連コードに触れる → P3 へ格上げ
│           └─ それ以外 → P4手動指示 (2エージェント)
│
│   └─ タイポ・設定値・ドキュメントのみ (50行以下)
│       └─ P5手動指示 (1エージェント)
│
├─ 本番リリースしたい
│   └─ 通常スプリントリリース
│       └─ /release-gate [バージョン] → P2 (6エージェント)
│   └─ 大型リリース・新規プロダクト・四半期レビュー
│       └─ /orchestrator [内容] → P1 (8エージェント)
│
└─ 緊急・障害
    └─ 本番障害が発生した
        └─ /incident-response [障害内容] → 緊急 (4~5エージェント)
    └─ 緊急Hotfixリリース
        └─ 3役割手動指示 (セキュリティ・技術・DevOps)
            └─ リリース後に事後 P2 レビューを必ず実施
```

P4 → P3 エスカレーション基準

P4 (コアレビュー) で以下に該当した場合は P3 へ格上げしてください：

チェック項目	該当する場合
認証・認可ロジックに触れている	P3 に格上げ (セキュリティ追加)
新しいUIコンポーネントを追加した	P3 に格上げ (UIデザイナー追加)
外部ライブラリを新規追加した	P3 に格上げ (法務追加)
想定外の設計変更が含まれていた	P3 に格上げ (技術責任者で再評価)
インフラ・CI/CDの変更が含まれる	P3 に格上げ (DevOps追加)

12. よくある質問 (FAQ)

Q1. スキルを配置したのにコマンドが表示されない

`.claude/skills/dev-team/` の直下に `SKILL.md` があるか確認してください。

例: `.claude/skills/dev-team/PRレビュー/SKILL.md`

また、Claude Code の再起動が必要な場合があります。

Q2. P1 フルチームを毎回使うとコストが高すぎる

日常の開発では P3~P4 を中心に使い、P1 は四半期リリースや重大な設計変更時のみに限定することを推奨します。

P5 (1エージェント) だけでも十分なケースが多くあります。

Q3. 役割エージェントを単独で呼びたい

各スキルは単独でも呼び出せます。

例: `/security-audit` (セキュリティ監査のみ)

`/cost-estimate` (コスト見積もりのみ)

`/test-plan` (テスト計画のみ)

Q4. 判定に納得できない場合はどうすればよいか

役割エージェントを個別に呼び出し、判断理由の詳細を確認してください。

例: `/code-review-security PR#42` のセキュリティ指摘の詳細を教えてください

Q5. 新しい役割を追加したい

`dev-team/` フォルダに新しい役割フォルダと `SKILL.md` を追加するだけです。

frontmatter に `context: fork` と `agent: general-purpose` を必ず記載してください。

オーケストレーターの `SKILL.md` にも委譲マップを追記することを忘れずに。

Q6. `context: fork` とはどういう意味か

サブエージェントを独立したコンテキスト (別セッション) で起動する設定です。

これにより各役割が互いに影響されず独立した判断を下せます。

オーケストレーターが全エージェントの結果を統合して返します。

Q7. `$ARGUMENTS` とはどういう意味か

スキル呼び出し時に渡した引数が展開される変数です。

例: `/pr-review PR#42` → スキル内の `$ARGUMENTS` が「PR#42」に置き換わります。

引数なしで呼ぶと `$ARGUMENTS` は空になります。

Q8. GitHub にプッシュしたスキルを別プロジェクトでも使いたい

`github.com/cowboybebopmasa-lab/skills-hinagata` をクローンして

各プロジェクトの `.claude/skills/` にシンボリックリンクを張るのが効率的です。

`git pull` でスキルの更新も全プロジェクトに即時反映されます。